LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 4 – ENKAPSULASI

NAMA: HUMAIRAH NURHAMIDAH

NIM: 103132400038

KELAS: PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Enkapsulasi adalah konsep dalam pemrograman berorientasi objek yang menyembunyikan data di dalam sebuah class sehingga tidak dapat diakses langsung dari luar, melainkan melalui method yang disediakan. Di Java, enkapsulasi diterapkan dengan *modifier*. *Access modifier* seperti `public`, `protected`, `private`, dan tanpa modifier mengatur siapa saja yang boleh mengakses suatu atribut atau method, sedangkan modifier `final` digunakan untuk membuat konstanta dan `static` untuk membuat variabel atau method milik class yang dapat dipakai tanpa membuat objek. Umumnya atribut dibuat `private` dan diakses lewat method `get` dan `set` yang bersifat `public`. Selain itu, *constructor* adalah method khusus yang namanya sama dengan nama class dan dipakai untuk membuat objek tanpa mengembalikan nilai. Sebuah class boleh memiliki beberapa constructor dengan parameter berbeda (*overloading*), dan Java akan memilih yang sesuai dengan parameter saat objek dibuat dengan kata kunci `new`. Terakhir, *package* digunakan untuk mengelompokkan class berdasarkan kategorinya agar program tertata rapi dan terhindar dari konflik nama, dengan `import` diperlukan jika class yang dipakai berada di package yang berbeda.

B. Guided

1. Manusia

```
package humaira.projectmodul04.guided;


public class Manusia {


//definisi atribut
    private String nama;
    private int umur;


    //4.2 constructor
    public Manusia(){};
    public Manusia(String nama){
        this.nama = nama;
```

```
    }
    public Manusia(String nama, int umur){
        this.nama = nama;
        this.umur = umur;
    }

    //method setter
    public void setNama(String a) {
        nama = a;
    }
    public String getNama() {
        return nama;
    }
    public void setUmur(int a) {
        umur = a;
    }
    public int getUmur() {
        return umur;
    }
}
```

Main

```
package humaira.projectmodul04.guided;

public class Main {
    public static void main(String[]args){
        Manusia arrMns[] = new Manusia[3]; // buat array of Object

        // constructor pertama
        Manusia objMns1 = new Manusia();
```

```
        objMns1.setNama("Markonah");
        objMns1.setUmur(76);
//constructor kedua

        Manusia objMns2 = new Manusia("Mat Conan");

//constructor ketiga
        Manusia objMns3 = new Manusia("Bajuri", 13);
        arrMns[0] = objMns1;
        arrMns[1] = objMns2;
        arrMns[2] = objMns3;
        for (int i = 0; i < 3; i++) {

            System.out.println("Nama : " + arrMns[i].getNama());
            System.out.println("Umur : " + arrMns[i].getUmur());
            System.out.println();
        }

    }
}
```

Output :

```
Nama : Markonah
Umur : 76

Nama : Mat Conan
Umur : 0

Nama : Bajuri
Umur : 13

------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

2. hargatoken

```
package humaira.projectmodul04.guided2;

public class hargatoken {

    public void info() {
        System.out.println("Harga Token");
        System.out.println("Rp. 20000");
```

```
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
        System.out.println("Rp. 200000");
        System.out.println("Rp. 500000");
    }
}
```

hargapulsa

```
package humaira.projectmodul04.guided2;

public class hargapulsa {

    public void info() {
        System.out.println("Harga Pulsa");
        System.out.println("Rp. 5000");
        System.out.println("Rp. 10000");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
    }
}
```

main

```
package humaira.projectmodul04.guided2;
public class Main {

    public static void main(String[] args) {
        hargatoken objectToken = new hargatoken();
        objectToken.info();

        hargapulsa objectPulsa = new hargapulsa();
        objectPulsa.info();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul04 ---
Harga Token
Rp. 20000
Rp. 50000
Rp. 100000
Rp. 200000
Rp. 500000
Harga Pulsa
Rp. 5000
Rp. 10000
Rp. 20000
Rp. 50000
Rp. 100000
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

C. Unguided

1. Soal Unguided

   Membuat program Java dengan struktur 3 package (model, laporan, dan main) untuk sistem pencatatan dataset: Dataset (package model): Menggunakan prinsip enkapsulasi, memiliki konstanta batas missing 5.0% counter total objek, 3 overloading constructor, serta method untuk menghitung persentase missing value dan status kebersihan data.

   LaporanDataset (package laporan): Memiliki method untuk mencetak format laporan informasi dataset secara rapi.

   Main (package main): Membuat 3 objek Dataset dengan constructor berbeda (tanpa parameter, 1 parameter, dan lengkap), menyimpannya dalam array, menampilkan laporannya menggunakan perulangan, serta mencetak total objek yang telah dibuat.

   modul

```
package model;

public class Dataset {
    // Atribut (private -> enkapsulasi)
    private String nama;
    private int jumlahBaris;
```

```
    private int jumlahKolom;
    private int jumlahMissing;

    // Konstanta batas missing (persen)
    public static final double BATAS_MISSING = 5.0;

    // Variabel milik class: total objek Dataset yang dibuat
    private static int totalDataset = 0;

    // Constructor 1: tanpa parameter
    public Dataset() {
        totalDataset++;
    }

    // Constructor 2: hanya nama
    public Dataset(String nama) {
        this.nama = nama;
        totalDataset++;
    }

    // Constructor 3: lengkap (lewat setter agar validasi tetap berlaku)
    public Dataset(String nama, int jumlahBaris, int jumlahKolom, int
jumlahMissing) {
        this.nama = nama;
        setJumlahBaris(jumlahBaris);
        setJumlahKolom(jumlahKolom);
        setJumlahMissing(jumlahMissing);
        totalDataset++;
    }

    // Method static untuk ambil total tanpa membuat objek
    public static int getTotalDataset() {
        return totalDataset;
    }

    // Getter
    public String getNama() {
        return nama;
    }

    public int getJumlahBaris() {
        return jumlahBaris;
    }

    public int getJumlahKolom() {
        return jumlahKolom;
```

```
    }

    public int getJumlahMissing() {
        return jumlahMissing;
    }

    // Setter (nilai negatif ditolak -> nilai tidak berubah)
    public void setNama(String nama) {
        this.nama = nama;
    }

    public void setJumlahBaris(int jumlahBaris) {
        if (jumlahBaris >= 0) {
            this.jumlahBaris = jumlahBaris;
        }
    }

    public void setJumlahKolom(int jumlahKolom) {
        if (jumlahKolom >= 0) {
            this.jumlahKolom = jumlahKolom;
        }
    }

    public void setJumlahMissing(int jumlahMissing) {
        if (jumlahMissing >= 0) {
            this.jumlahMissing = jumlahMissing;
        }
    }

    // Persentase sel kosong terhadap total sel
    public double getPersentaseMissing() {
        int totalSel = jumlahBaris * jumlahKolom;
        if (totalSel == 0) {
            return 0;
        }
        return (double) jumlahMissing / totalSel * 100;
    }

    // true jika persentase missing melebihi batas
    public boolean perluDibersihkan() {
        return getPersentaseMissing() > BATAS_MISSING;
    }
}
```

laporan

```
package laporan;


import model.Dataset;


public class LaporanDataset {


    public void cetak(Dataset d) {
        System.out.println("=== Laporan Dataset ===");
        System.out.println("Nama         : " + d.getNama());
        System.out.println("Jumlah Baris : " + d.getJumlahBaris());
        System.out.println("Jumlah Kolom : " + d.getJumlahKolom());
        System.out.printf("Missing      : %d sel (%.2f%%)%n",
                d.getJumlahMissing(), d.getPersentaseMissing());
        System.out.println("Status       : "
                + (d.perluDibersihkan() ? "Perlu dibersihkan" : "Bersih"));
        System.out.println();
    }
}
```

main

```
package main;

import model.Dataset;
import laporan.LaporanDataset;

public class Main {
    public static void main(String[] args) {
        // Objek 1: constructor tanpa parameter, lalu isi dengan setter
        Dataset d1 = new Dataset();
        d1.setNama("Titanic");
        d1.setJumlahBaris(891);
        d1.setJumlahKolom(12);
```

```
        d1.setJumlahMissing(866);

        // Objek 2: constructor 1 parameter
        Dataset d2 = new Dataset("Wine Quality");

        // Objek 3: constructor lengkap
        Dataset d3 = new Dataset("Iris", 150, 5, 0);

        // Simpan dalam array
        Dataset[] daftar = {d1, d2, d3};

        // Cetak laporan dengan perulangan
        LaporanDataset laporan = new LaporanDataset();
        for (Dataset d : daftar) {
            laporan.cetak(d);
        }

        System.out.println("Total dataset dibuat : " + Dataset.getTotalDataset());
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul04 ---
=== Laporan Dataset ===
Nama         : Titanic
Jumlah Baris : 891
Jumlah Kolom : 12
Missing      : 866 sel (8,10%)
Status       : Perlu dibersihkan

=== Laporan Dataset ===
Nama         : Wine Quality
Jumlah Baris : 0
Jumlah Kolom : 0
Missing      : 0 sel (0,00%)
Status       : Bersih

=== Laporan Dataset ===
Nama         : Iris
Jumlah Baris : 150
Jumlah Kolom : 5
Missing      : 0 sel (0,00%)
Status       : Bersih

Total dataset dibuat : 3
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Penjelasan :

Pada praktikum ini enkapsulasi diterapkan dengan menjadikan semua atribut class `Dataset` bersifat `private`, sehingga data hanya bisa diakses lewat getter dan diubah lewat setter yang sudah diberi validasi agar nilai tidak valid, seperti angka negatif, tidak merusak data. Penggunaan konstanta `BATAS_MISSING` membuat batas kualitas data terdefinisi jelas dan tidak mudah berubah, sedangkan variabel dan method `static` memungkinkan penghitungan total objek yang dibuat tanpa perlu membuat objek terlebih dahulu. Tiga constructor yang berbeda (overloading) memberi fleksibilitas dalam membuat objek, baik kosong, hanya dengan nama, maupun lengkap. Pemisahan kode ke dalam tiga package, yaitu `model`, `laporan`, dan `main`, membuat tanggung jawab tiap class lebih jelas dan program lebih terstruktur serta mudah dirawat. Secara keseluruhan, program ini menunjukkan bahwa enkapsulasi membantu menjaga keamanan dan konsistensi data sekaligus membuat kode lebih rapi dan mudah dikembangkan.

D. Kesimpulan

Dari Modul 4 dapat disimpulkan bahwa enkapsulasi merupakan konsep penting untuk menjaga keamanan dan keteraturan data dalam pemrograman berorientasi objek. Melalui access modifier, atribut dapat disembunyikan dengan `private` dan hanya diakses lewat method `get` dan `set` yang bersifat `public`, sehingga data tidak diubah sembarangan dari luar class. Modifier lain seperti `final` dan `static` memperluas pengendalian tersebut, misalnya untuk membuat konstanta dan variabel atau method milik class. Constructor membantu proses pembuatan objek, dan dengan overloading sebuah class dapat dibuat dengan beberapa cara sesuai kebutuhan data awal yang tersedia. Package memungkinkan class dikelompokkan sesuai fungsinya dan menghindari bentrok nama, sehingga program menjadi lebih terstruktur dan mudah dikembangkan. Dengan menerapkan ketiga konsep ini secara bersamaan, program yang dihasilkan menjadi lebih aman, rapi, dan mudah dirawat.